CSD-380: DevOps
Module 6: Continuous Integration and Low-Risk Releases
Assignment 2: Case Study: Strangler Pattern at Blackboard Learn (2011)
Isaac Ellingson
7/5/2026

Case Study: Strangler Fig Pattern at Blackboard

The Problem:

The Blackboard web application had been growing substantially since its creation in 1997. Looking at its own SLOC graph, it had a sharp uptick in code complexity in 2009. This seemed to have pushed the team past its limit, and code commit frequency fell steadily over the following year even as code complexity kept growing.

This was visible to the development team and to customers, with slow, fragile testing automation, poor customer outcomes, and long feature lead times.

A Solution:

In 2012, they envisioned a kind of microservice that they called Building Blocks. This must have involved creating a microservices architecture plan, because these modules had a fixed, known API, which allowed them to be decoupled from the original monolithic app. Per the usual strangler fig pattern, once these services were running, functionality was forwarded and migrated to the new service, and deprecated within the monolith for eventual removal.

The benefits of this are obvious: Well-architected modularity allows independent teams to work efficiently, without having to worry about concerns external to their "Building Block". The entire system doesn't have to suddenly go from "A Monolith" to "A Cloud of Microservices", which would require shutting the system down to rebuild it from scratch. And once the entire system is respecting the separation of concerns / single responsibility principle, a given module can be enhanced or redesigned without disrupting the rest of the codebase, as long as its API remains.

This approach has some risks - the eventual consistency that microservices embody is not right for every project. But in this case, they show an explosion in commit frequency and code complexity, *suggesting* but not proving that the team and application thrived under these conditions. The case study also mentions that they updated their build process, and implies that the faster code turnaround contributed to faster feedback and better quality.

Lessons Learned:

Clearly you don't have to pick monolith or microservices at the start of the project. And clearly the strangler fig pattern can be the right answer for *some teams*. One metric I'd love to see here that would really drive home the point is code churn. More SLOC is not often better for a project - and in fact I think we've all seen that the Blackboard app has some key UI/UX issues to this day. I'm still missing important features I enjoyed in Canvas during my Associate's program. So I'm not fully sold on "we did strangler fig and it solved everything". It didn't.

But, look at that SLOC drop in late 2012. I'm willing to bet that's where they retired a lot of monolith code. That's probably the real story here, that each individual piece's code complexity - the SLOC for *each module*, that *each team has to keep in their head* when they're working on something, that suddenly dropped. Combined with the ability to look carefully and clearly at what their system needs in terms of division of responsibilities, that created a productivity explosion *of some kind*.

So there are things I'd like to see to make a stronger case for it, but I feel we can still gain some good intuition from this study about where and how to apply the pattern.